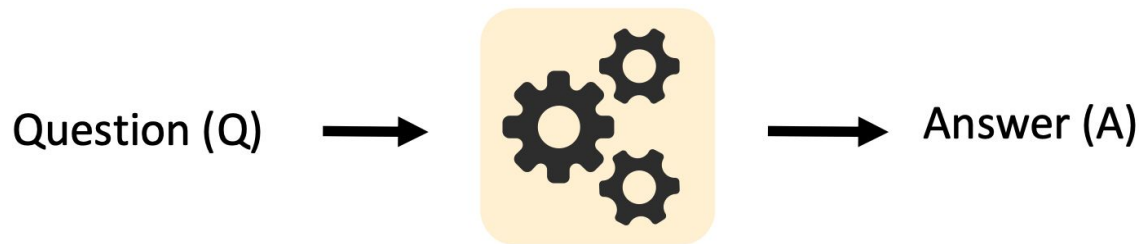


Chapter 8: Question Answering

Question answering

What is question answering?

The goal of question answering is to build systems that automatically answer questions posed by humans in a natural language



Question answering: a taxonomy

- What information source does a system build on?
 - A text passage, all Web documents, knowledge bases, tables, images..
- Question type
 - Factoid vs non-factoid, open-domain vs closed-domain, simple vs compositional, ..
- Answer type
 - A short segment of text, a paragraph, a list, yes/no, ...

Lots of practical applications

2011: IBM Watson



Lots of practical applications



what is IBM Watson

All Images Videos News Short videos Shopping Forums : More

◆ AI Overview

IBM Watson is a powerful AI platform and suite of services that leverages natural language processing and machine learning to help businesses understand data, interact with people, and make better decisions. [↗](#)



Here's a more detailed breakdown:

Key Features and Capabilities:

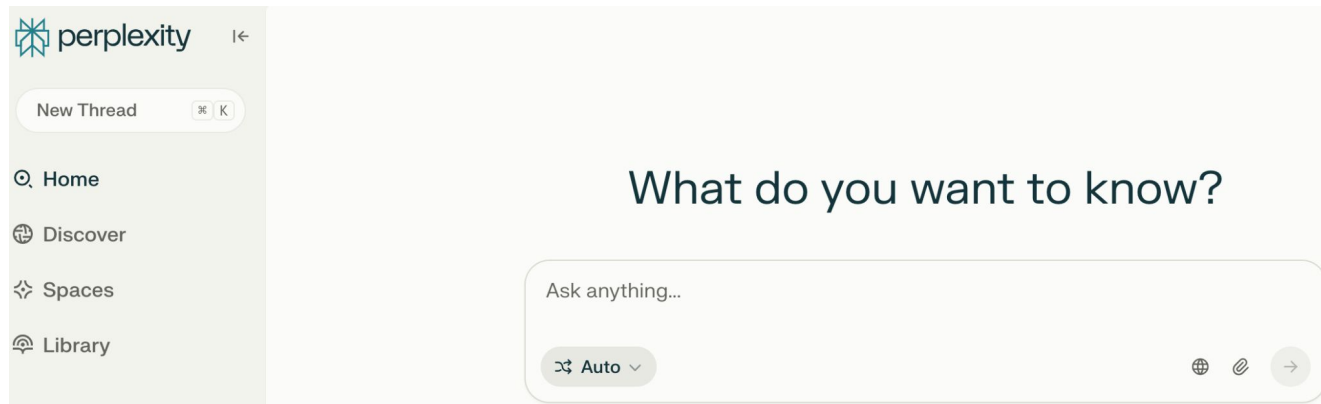
Natural Language Processing:

Watson can understand and respond to human language, allowing for more natural and intuitive interactions. [↗](#)

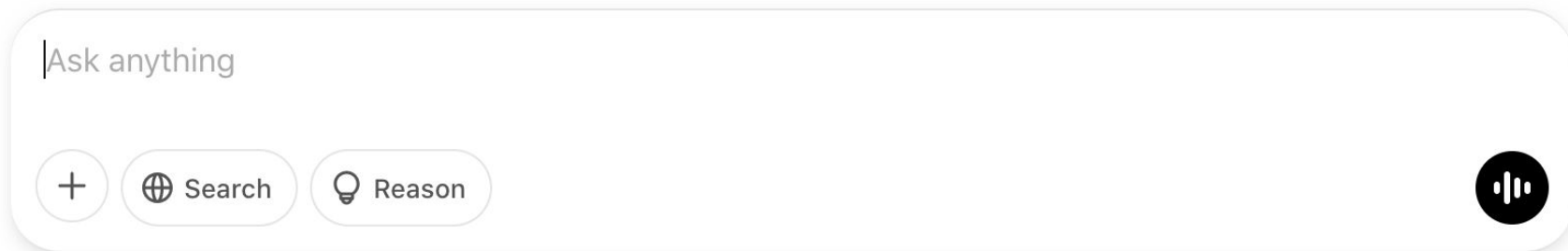
Data Analytics:

It can analyze vast amounts of data to identify patterns, trends, and insights that might otherwise be hidden. [↗](#)

Lots of practical applications



What can I help with?



Closed book

Different types of QA

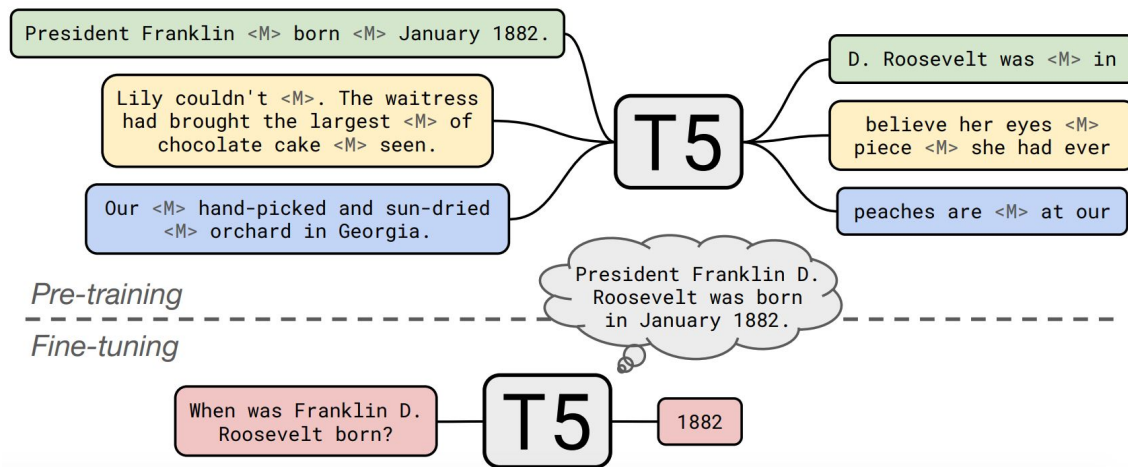
- Closed book
 - 'parametric' QA
- Open book
 - Reading comprehension
 - RAG

Closed book QA

- Closed book: answering the question with no access to external information
- Q&A done just by ‘predicting the next word’
 - Stanford University is located in ...

Closed book QA

- Large language models can do open-domain QA well
 - Initially – for models like T5, people discovered LMs can directly answer questions



Closed book QA

- LMs have remarkable knowledge about the world
- Closed book QA as a capability test of LMs
- Many popular benchmarks are partially knowledge tests
 - ARC-C – A dataset for commonsense reasoning.
 - MMLU – Covers a wide range of subjects from science to humanities.
 - GPQA – Tests knowledge retrieval and reasoning.

Closed book QA

- LLMs are remarkably good at many things – even closed-book QA .. But they aggressively make things up for things they don't know: Hallucinations
- How to deal with it?
 - 'Backing off'
 - Adding new knowledge to a model

Closed book QA

- How to deal with it?
 - 'Backing off'
 - Estimate the confidence of a statement
 - Abstain whenever the confidence is too low

Closed book QA

- How to deal with it?
 - Adding new knowledge to a model
 - continue to do pretraining with domain-specific data
 - continue to do pretraining with synthetic data

Open book QA

Open book QA - Reading comprehension

Reading comprehension = comprehend a passage of text and answer questions about its content

$(P, Q) \rightarrow A$

Open book QA - Reading comprehension

Tesla was the fourth of five children. He had an older brother named Dane and three sisters, Milka, Angelina and Marica. Dane was killed in a horse-riding accident when Nikola was five. In 1861, Tesla attended the "Lower" or "Primary" School in Smiljan where he studied German, arithmetic, and religion. In 1862, the Tesla family moved to Gospić, Austrian Empire, where Tesla's father worked as a pastor. Nikola completed "Lower" or "Primary" School, followed by the "Lower Real Gymnasium" or "Normal School."

Q: What language did Tesla study while in school?

A: German

Why do we care about this problem?

Useful for many practical applications

- Reading comprehension is an important testbed for evaluating how well computer systems understand human language
- Many other NLP tasks can be reduced to a reading comprehension problem:
 - Information extraction
 - Semantic role labeling

Question answering datasets

SQuAD (Stanford Question Answering Dataset)

TriviaQA: Questions and answers by trivia enthusiasts.

Independently collected web paragraphs that contain the answer and seem to discuss question, but no human verification that paragraph supports answer to question

Question answering datasets

Natural Questions: Question drawn from frequently asked Google search questions. Answers from Wikipedia paragraphs. Answer can be substring, yes, no, or NOT_PRESENT. Verified by human annotation.

Question answering datasets

HotpotQA: Constructed questions to be answered from the whole of Wikipedia which involve getting information from two pages to answer a multistep query:

Q: Which novel by the author of “Armada” will be adapted as a feature film by Steven Spielberg?

A: Ready Player One

Open-domain question answering

- Different from reading comprehension, we don't assume a given passage
- Instead, we only have access to a large collection of documents (e.g., Wikipedia). We don't know where the answer is located, and the goal is to return the answer for any open-domain questions.
- Much more challenging and a more practical problem!

Retrieval augmentation

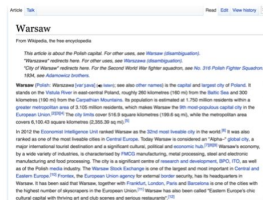
Open-domain QA

SQuAD, TREC, WebQuestions, WikiMovies

Q: How many of Warsaw's inhabitants spoke Polish in 1933?

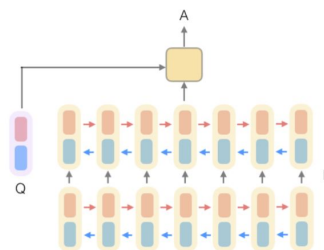


**Document
Retriever**



**Document
Reader**

833,500



Chen et al., 2017. Reading Wikipedia to Answer Open-domain Questions

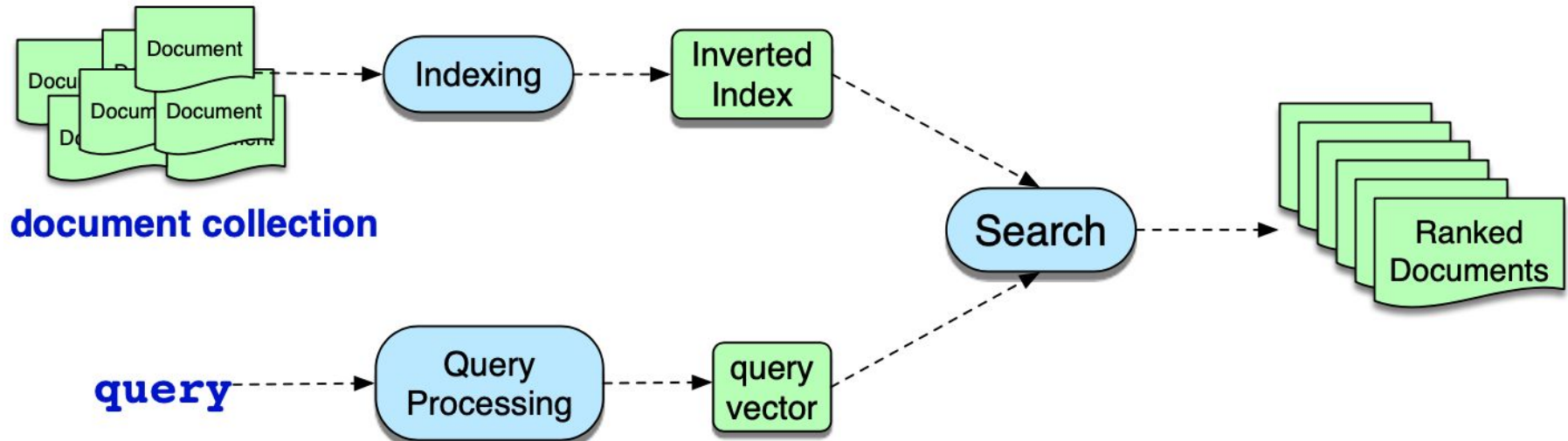
DrQA is a system for reading comprehension applied to open-domain question answering.

Using retrieval to overcome LMs' shortcomings

- Instead of asking the LM to memorize everything, can we provide the LM with relevant and useful content just-in-time?
- Retrieval / search is a common mechanism for identifying such relevant information.
 - **Dynamic:** it's easy to update / add documents to your retrieval system
 - **Interpretable:** LM can generate pointers to retrieved documents that support human verification of its generations (citations)

Using retrieval to overcome LMs' shortcomings

Information Retrieval (or search engine): the retrieval of all IR manner of media based on user information needs.



Information retrieval

Term weighting and document scoring: tf-idf

Term frequency **tf**

$$\text{tf}_{t,d} = \begin{cases} 1 + \log_{10} \text{count}(t,d) & \text{if } \text{count}(t,d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

Document frequency **df_t** of a term **t**

$$\text{idf}_t = \log_{10} \frac{N}{\text{df}_t}$$

27 $\text{tf-idf}(t,d) = \text{tf}_{t,d} \cdot \text{idf}_t$

Information retrieval

Term weighting and document scoring: tf-idf

Document Scoring

$$\text{score}(q, d) = \cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{|\mathbf{q}| |\mathbf{d}|}$$

Example:

Query:	sweet love
Doc 1:	Sweet sweet nurse! Love?
Doc 2:	Sweet sorrow
Doc 3:	How sweet is love?
Doc 4:	Nurse!

Query:	sweet love
Doc 1:	Sweet sweet nurse! Love?
Doc 2:	Sweet sorrow
Doc 3:	How sweet is love?
Doc 4:	Nurse!

vocab:

how is love nurse sorrow sweet

q [tfidf(how) tfidf(is) tfidf(love) ...]

d1 [tfidf(how) tfidf(is) tfidf(love) ...]

$$\text{tf}_{t,d} = \begin{cases} 1 + \log_{10} \text{count}(t,d) \\ 0 \end{cases}$$

$$\text{score}(q,d) = \cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{|\mathbf{q}| |\mathbf{d}|}$$

$$\text{idf}_t = \log_{10} \frac{N}{\text{df}_t}$$

q:	[0	0	0.301	0	0	0.124]:
d1	[0	0	0.301	0.301	0	0.162],
d2	[0	0	0	0	0.124	0.301]
d3	[0.602	0.602	0.301	0	0	0.12]
d4	[0	0	0	0.301	0	0]

Query:	sweet love
Doc 1:	Sweet sweet nurse! Love?
Doc 2:	Sweet sorrow
Doc 3:	How sweet is love?
Doc 4:	Nurse!

vocab:

how is love nurse sorrow sweet

q [tfidf(how) tfidf(is) tfidf(love) ...]

d1 [tfidf(how) tfidf(is) tfidf(love) ...]

$$\text{score}(q, d) = \cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{|\mathbf{q}| |\mathbf{d}|}$$

q:	[0	0	0.301	0	0	<u>0.124</u>]	:
d1	[0	0	0.301	0.301	0	0.162]	: <u>0.746</u>
d2	[0	0	0	0	0.124	<u>0.301</u>]	: <u>0.352</u>
d3	[0.602	0.602	0.301	0	0	0.12]	: 0.355
d4	[0	0	0	0.301	0	0]	: 0

Information retrieval

Term weighting and document scoring: BM25

BM25 (best match) adds two parameters:

k_1 : adjust the balance between TF and IDF

b : controls the importance of document length normalization

The BM25 score of a document d given a query q :

$$\sum_{t \in q} \log \left[\frac{N}{\text{df}_t} \right] \cdot \frac{(k_1 + 1) \text{tf}_{td}}{k_1 ((1 - b) + b \times (L_d / L_{\text{ave}})) + \text{tf}_{td}}$$

Query: sweet love

Doc 1: Sweet sweet nurse! Love?

Doc 2: Sweet sorrow

Doc 3: How sweet is love?

Doc 4: Nurse!

vocab:

how is love nurse sorrow sweet

q [tfidf(how) tfidf(is) tfidf(love) ...]

d1 [tfidf(how) tfidf(is) tfidf(love) ...]

$$\sum_{t \in q} \log \left[\frac{N}{\text{df}_t} \right] \cdot \frac{(k_1 + 1) \text{tf}_{td}}{k_1 ((1 - b) + b \times (L_d / L_{\text{ave}})) + \text{tf}_{td}}$$

4 (4, 2, 4, 1): 2.75

d1:

sweet: 0.122

$\log(4/3) * (1.5+1) * (1+\log 2) /$

$1.5 * (0.25 + 0.75 * 4/2.75) + (1+\log 2)$

Query: sweet love

Doc 1: Sweet sweet nurse! Love?

Doc 2: Sweet sorrow

Doc 3: How sweet is love?

Doc 4: Nurse!

vocab:

how is love nurse sorrow sweet

q [tfidf(how) tfidf(is) tfidf(love) ...]

d1 [tfidf(how) tfidf(is) tfidf(love) ...]

$$\sum_{t \in q} \log \left[\frac{N}{\text{df}_t} \right] \cdot \frac{(k_1 + 1) \text{tf}_{td}}{k_1 ((1 - b) + b \times (L_d / L_{\text{ave}})) + \text{tf}_{td}}$$

4 (4, 2, 4, 1): 2.75

d1: (0.122+0.249) = 0.371

love: 0.249

$\log(4/2) * (1.5+1) * (1+\log 1) /$

$1.5 * (0.25 + 0.75 * 4/2.75) + (1+\log 1)$

Information retrieval - Inverted Index

The basic search problem in IR is thus to find all documents $d \in C$ that contain a term $q \in Q$.

The data structure for this task is the **inverted index**

Query: sweet love

Doc 1: Sweet sweet nurse! Love?

Doc 2: Sweet sorrow

Doc 3: How sweet is love?

Doc 4: Nurse!

trieval - Inverted Index

inverted index consists of two parts:

dictionary: a list of terms, each term points to its postings list

postings: a list of document IDs associated with each term

The diagram shows an inverted index table with terms in the first column, their document frequency df_t in curly braces, and their postings lists in the second column. A callout box labeled df_t points to the curly braces, and another callout box labeled tf_t points to the first document ID in the postings list for the term 'how'.

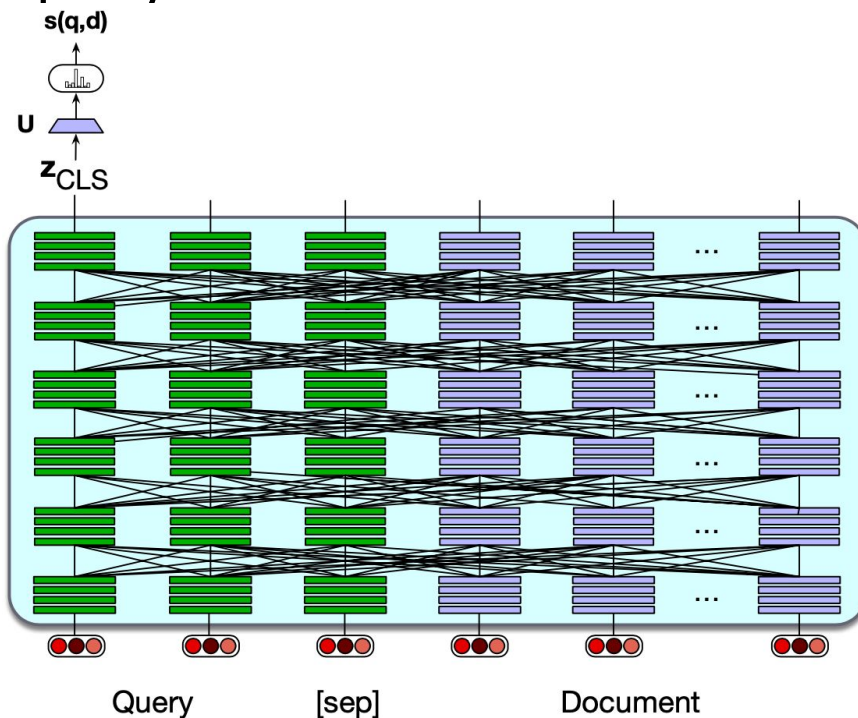
how	$\{1\}$	$\rightarrow 3 [1]$
is	$\{1\}$	$\rightarrow 3 [1]$
love	$\{2\}$	$\rightarrow 1 [1] \rightarrow 3 [1]$
nurse	$\{2\}$	$\rightarrow 1 [1] \rightarrow 4 [1]$
sorry	$\{1\}$	$\rightarrow 2 [1]$
sweet	$\{3\}$	$\rightarrow 1 [2] \rightarrow 2 [1] \rightarrow 3 [1]$

Information Retrieval with Dense Vectors

- The classic tf-idf or BM25 algorithms for IR work only if there is exact overlap of words between the query and document:
vocabulary mismatch problem
- Solution: instead of (sparse) word-count vectors, using (dense) embeddings

Information Retrieval with Dense Vectors

Present both the query and the document to a single encoder:

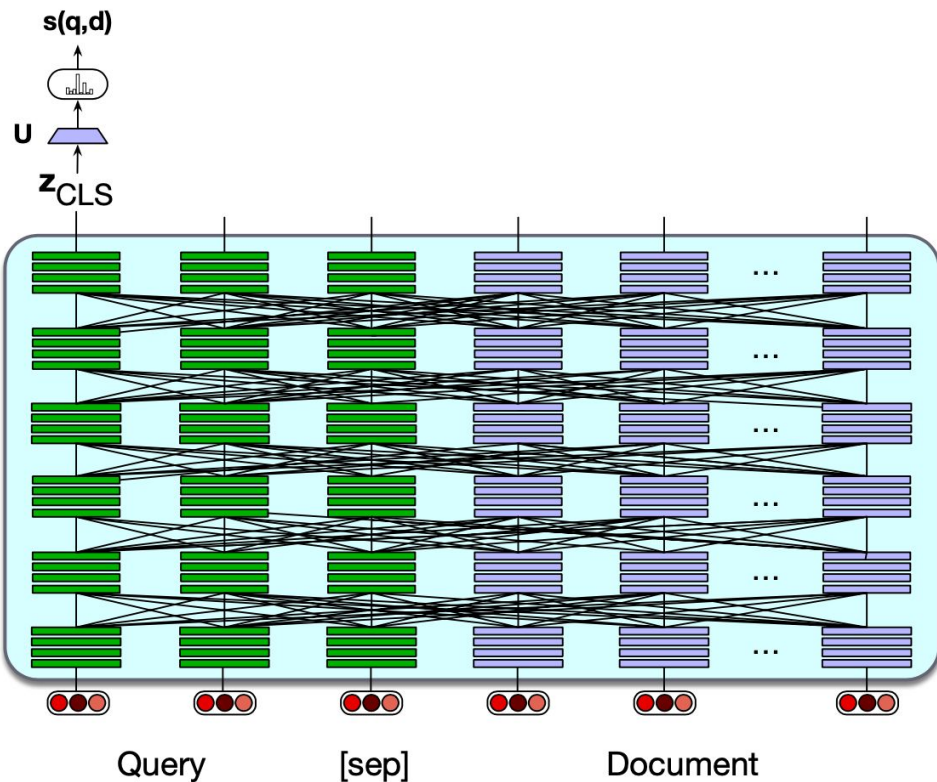


Information Retrieval with Dense Vectors

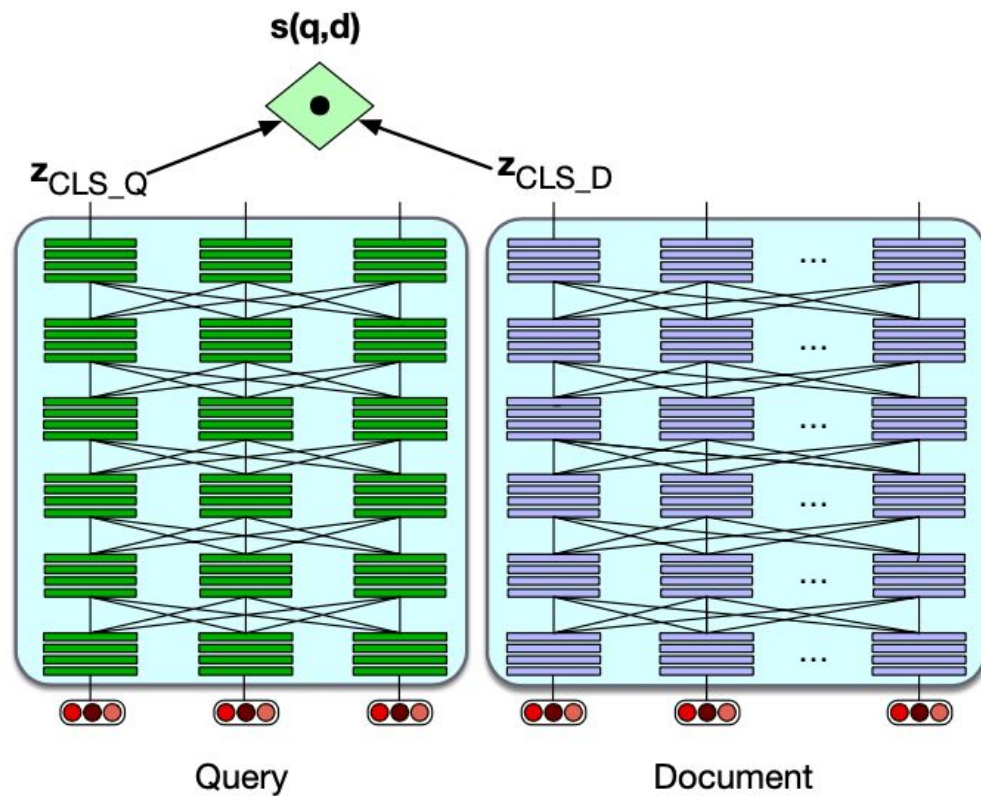
Present both the query and the document to a single encoder

$$\mathbf{z} = \text{BERT}(q; [\text{SEP}]; d) [\text{CLS}]$$

$$\text{score}(q, d) = \text{softmax}(\mathbf{U}(\mathbf{z}))$$



Information Retrieval with Dense Vectors



Use separate encoders for query and document, and use the dot product between CLS token outputs for the query and document as the score.

$$\mathbf{z}_q = \text{BERT}_Q(q) [\text{CLS}]$$

$$\mathbf{z}_d = \text{BERT}_D(d) [\text{CLS}]$$

$$\text{score}(q, d) = \mathbf{z}_q \cdot \mathbf{z}_d$$

Retrieval Augmented Generation (RAG)

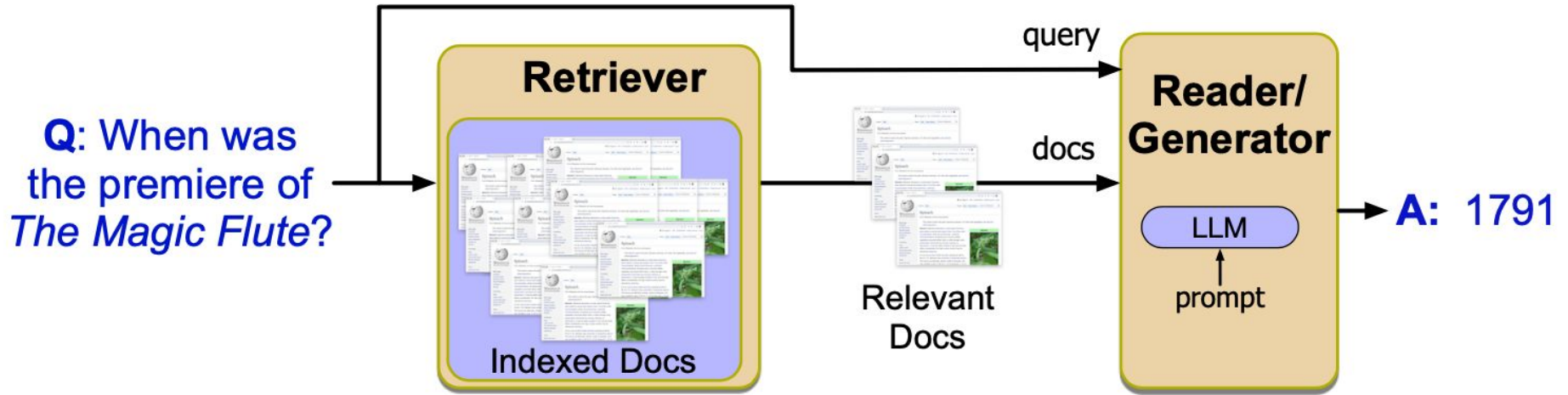


Figure 14.9 Retrieval-based question answering has two stages: **retrieval**, which returns relevant documents from the collection, and **reading**, in which an LLM **generates** answers given the documents as a prompt.

Evaluating Question Answering

Evaluating Question Answering

For multiple choice questions like in MMLU, we report exact match:

- Exact match: The % of predicted answers that match the gold answer exactly.

Evaluating Question Answering

For questions with free text answers, F1 score is used:

- The average token overlap between predicted and gold answers.
- Treat the prediction and gold as a bag of tokens, and compute F1 for each question, then return the average F1 over all questions.

Evaluating Question Answering

QA systems give multiple ranked answers, we evaluated by using mean reciprocal rank, or MRR

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

- $|Q|$ is the total number of queries.
- rank_i is the position of the first relevant result for query i .

Evaluating Question Answering

Reciprocal Rank (RR):

- For each query, the RR is calculated. If the first relevant result is at rank position 'k', then the RR is $1/k$.
- If there are no relevant results in the ranked list, the RR is 0.

Evaluating Question Answering

3 queries and their ranks of the first relevant documents are:

- Query 1: First relevant result at **position 2** → Reciprocal rank = $1/2=0.5$
- Query 2: First relevant result at **position 3** → Reciprocal rank = $1/3\approx0.333$
- Query 3: First relevant result at **position 1** → Reciprocal rank = $1/1=1.0$

$$\text{MRR} = 1/3 * (0.5+0.333+1.0) = 0.611$$

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$